



សាកលវិទ្យាល័យនំរតុន
NORTON UNIVERSITY



User Registration System

2025 – 2026
Y4 – DCS – NU



User Registration Using Flask and SQLite3 (Cont.)

By: SEK SOCHEAT

Advisor to DCS and Lecturer

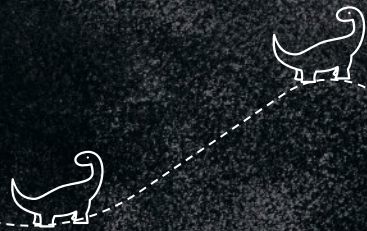
Mobile: 017 879 967

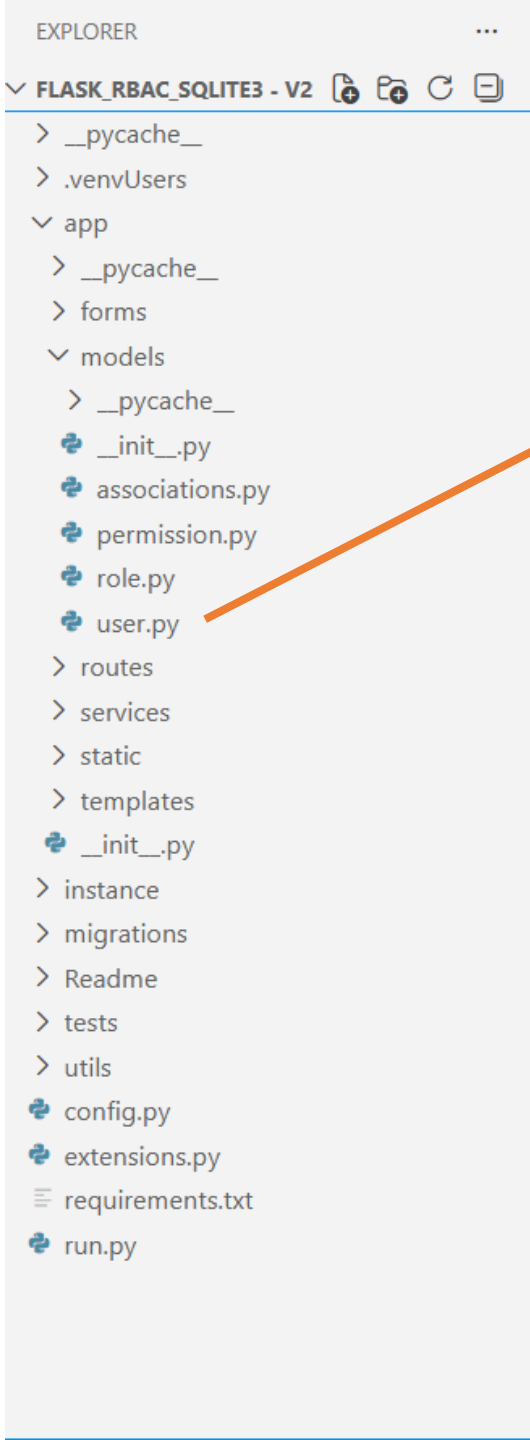
Email: socheatsek@norton-u.com

socheat.sek@gmail.com



1 — Update Python codes in models/





```
app > models > user.py > ...
1  from datetime import datetime
2  from werkzeug.security import generate_password_hash, check_password_hash
3  from flask_login import UserMixin
4  from extensions import db
5  from app.models.associations import user_roles
6
7  class User(UserMixin, db.Model):
8      __tablename__ = "users"
9
10     id = db.Column(db.Integer, primary_key=True)
11     username = db.Column(db.String(80), unique=True, nullable=False)
12     email = db.Column(db.String(120), unique=True, nullable=False)
13     full_name = db.Column(db.String(120), nullable=False)
14     is_active = db.Column(db.Boolean, default=True, nullable=False)
15     password_hash = db.Column(db.String(255), nullable=False)
16     created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
17     updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False)
18
19     roles = db.relationship("Role", secondary=user_roles, back_populates="users")
20
21     def set_password(self, password: str) -> None:
22         self.password_hash = generate_password_hash(password)
23
24     def check_password(self, password: str) -> bool:
25         return check_password_hash(self.password_hash, password)
26
27     def has_role(self, role_name: str) -> bool:
28         return any(role.name == role_name for role in self.roles)
29
30     def get_permission_codes(self) -> set[str]:
31         return {perm.code for role in self.roles for perm in role.permissions}
32
33     def has_permission(self, permission_code: str) -> bool:
34         return permission_code in self.get_permission_codes()
35
36     def __repr__(self) -> str:
37         return f"<User {self.username}>"
```

> __pycache__
> .venvUsers
v app
 > __pycache__
 > forms
 v models
 > __pycache__
 • __init__.py
 • associations.py
 • permission.py
 • role.py
 • user.py
 > routes
 > services
 > static
 > templates
 • __init__.py
 > instance
 > migrations
 > Readme
 > tests
 > utils
• config.py
• extensions.py
≡ requirements.txt
• run.py

app > models > role.py > ...

```
1  from datetime import datetime
2  from extensions import db
3  from app.models.associations import user_roles, role_permissions
4
5  class Role(db.Model):
6      __tablename__ = "roles"
7
8      id = db.Column(db.Integer, primary_key=True)
9      name = db.Column(db.String(80), unique=True, nullable=False)
10     description = db.Column(db.String(255))
11
12     created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
13     updated_at = db.Column(
14         db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False
15     )
16
17     users = db.relationship("User", secondary=user_roles, back_populates="roles")
18     permissions = db.relationship("Permission", secondary=role_permissions, back_populates="roles")
19
20     def has_permission(self, permission_code: str) -> bool:
21         return any(p.code == permission_code for p in self.permissions)
22
23     def __repr__(self) -> str:
24         return f"<Role {self.name}>"
```


> __pycache__
> .venvUsers
v app
 > __pycache__
 > forms
 v models
 > __pycache__
 • __init__.py
 • associations.py
 • permission.py
 • role.py
 • user.py
 > routes
 > services
 > static
 > templates
 • __init__.py
 > instance
 > migrations
 > Readme
 > tests
 > utils
• config.py
• extensions.py
≡ requirements.txt
• run.py

app > models > permission.py > ...

```
1  from datetime import datetime
2  from extensions import db
3  from app.models.associations import role_permissions
4
5  class Permission(db.Model):
6      __tablename__ = "permissions"
7
8      id = db.Column(db.Integer, primary_key=True)
9      code = db.Column(db.String(64), unique=True, nullable=False, index=True)
10     name = db.Column(db.String(120), nullable=False)
11     description = db.Column(db.String(255))
12     module = db.Column(db.String(80), nullable=False, default="General")
13     created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
14     updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False)
15
16     roles = db.relationship("Role", secondary=role_permissions, back_populates="permissions")
17
18     def __repr__(self) -> str:
19         return f"<Permission {self.code}>"
```

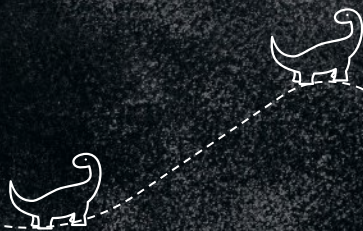
> __pycache__
> .venvUsers
▼ app
 > __pycache__
 > forms
 ▼ models
 > __pycache__
 • __init__.py
 • associations.py
 • permission.py
 • role.py
 • user.py
 > routes
 > services
 > static
 > templates
 • __init__.py
> instance
> migrations
> Readme
> tests
> utils
• config.py
• extensions.py
≡ requirements.txt
• run.py

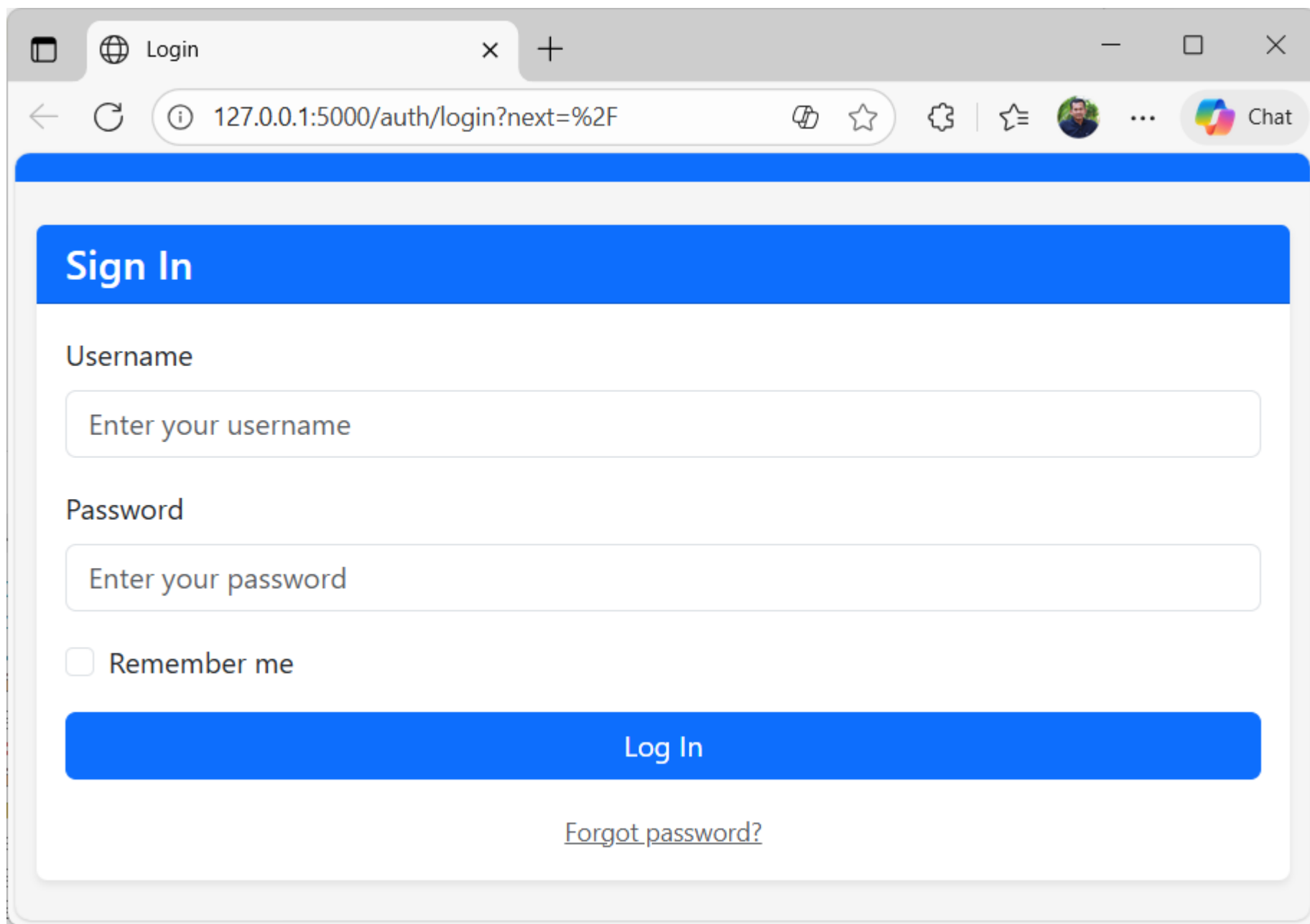
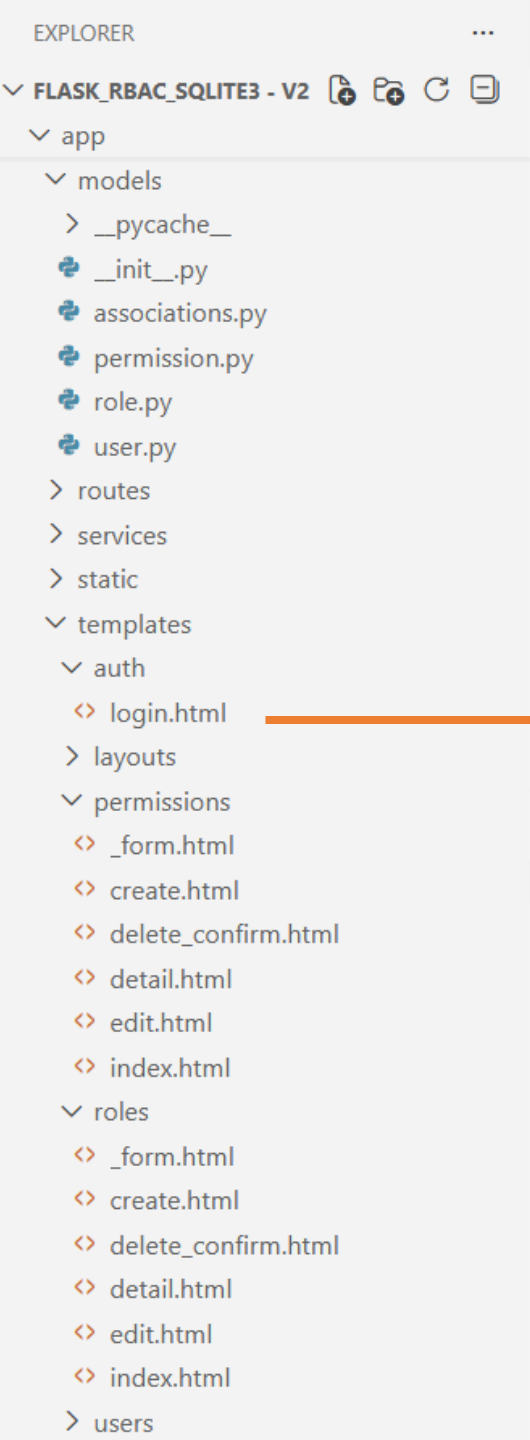
app > models > associations.py > ...

```
1  from extensions import db
2
3  user_roles = db.Table(
4      "user_roles",
5      db.Column("user_id", db.Integer, db.ForeignKey("users.id"), primary_key=True),
6      db.Column("role_id", db.Integer, db.ForeignKey("roles.id"), primary_key=True),
7  )
8
9  role_permissions = db.Table(
10     "role_permissions",
11     db.Column("role_id", db.Integer, db.ForeignKey("roles.id"), primary_key=True),
12     db.Column("permission_id", db.Integer, db.ForeignKey("permissions.id"), primary_key=True),
13 )
```



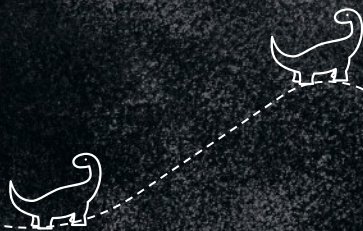
2 — GUI in templates/auth

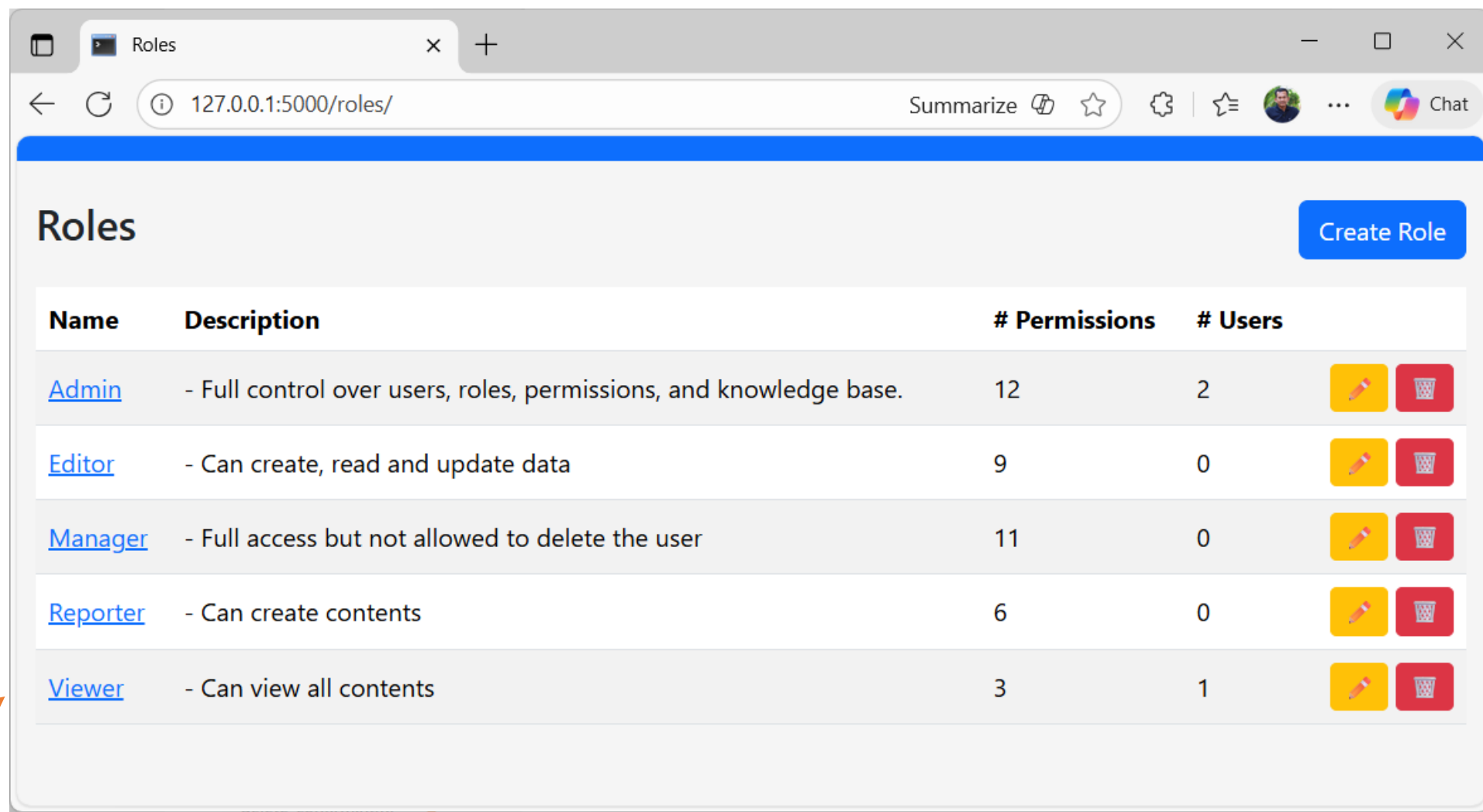
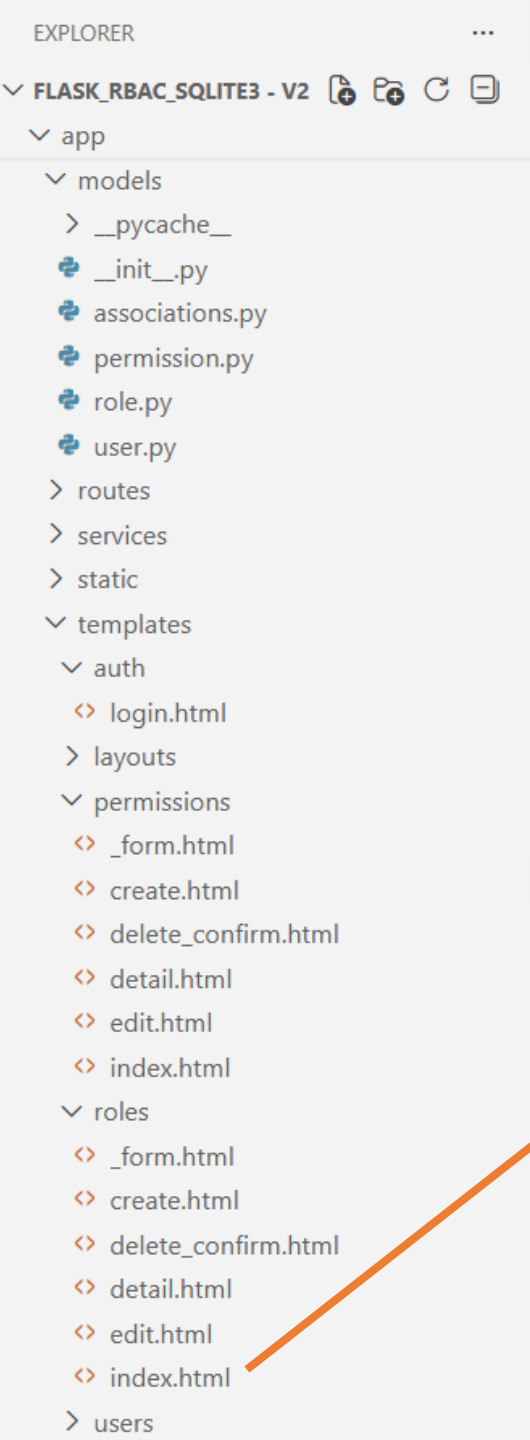


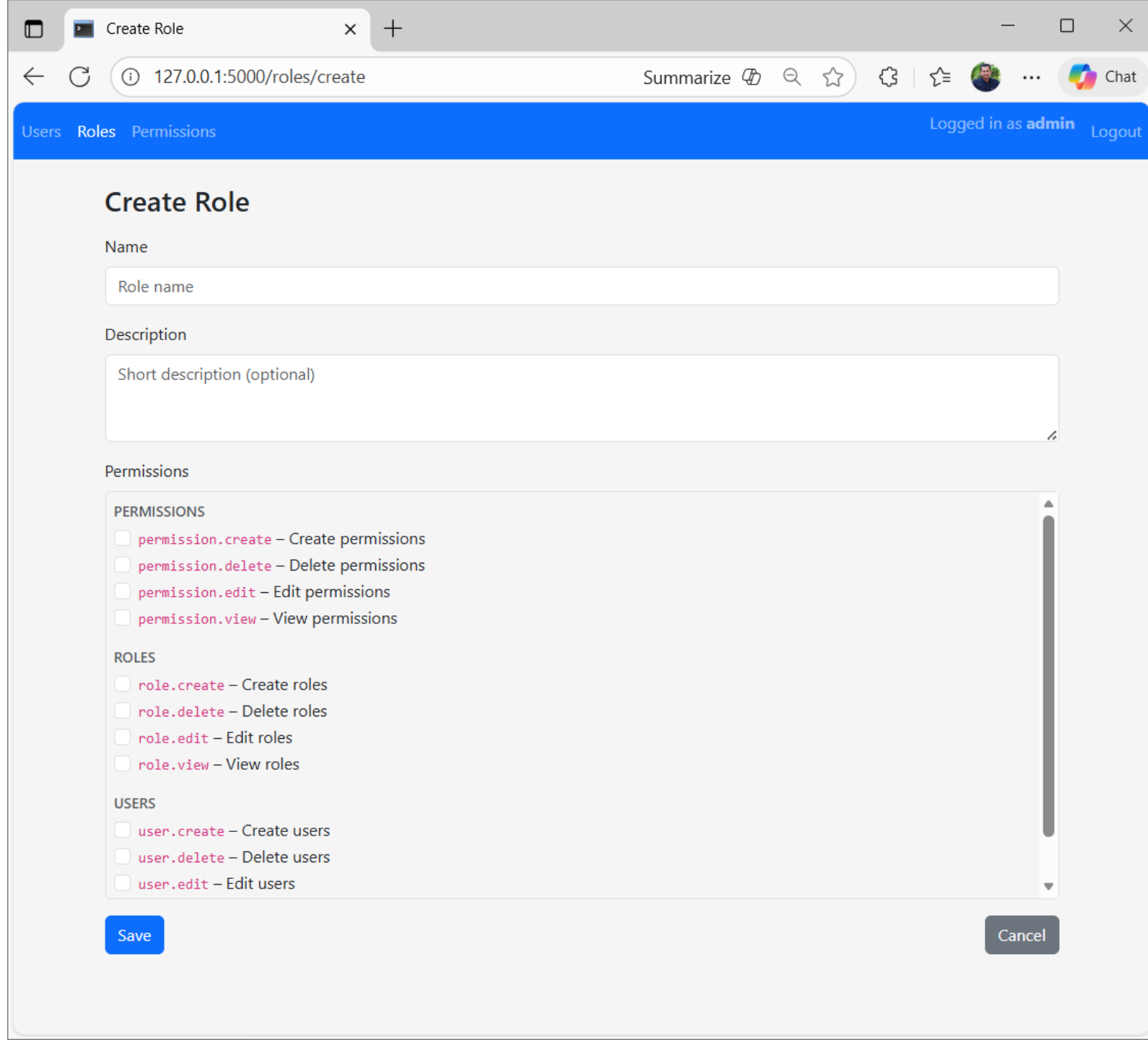
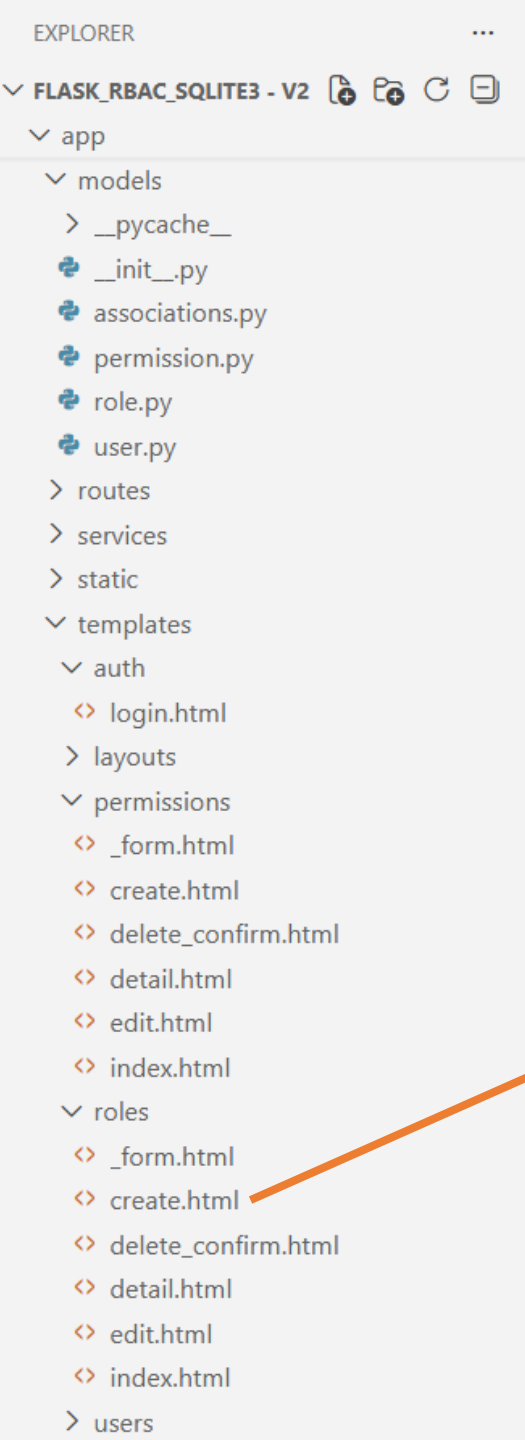


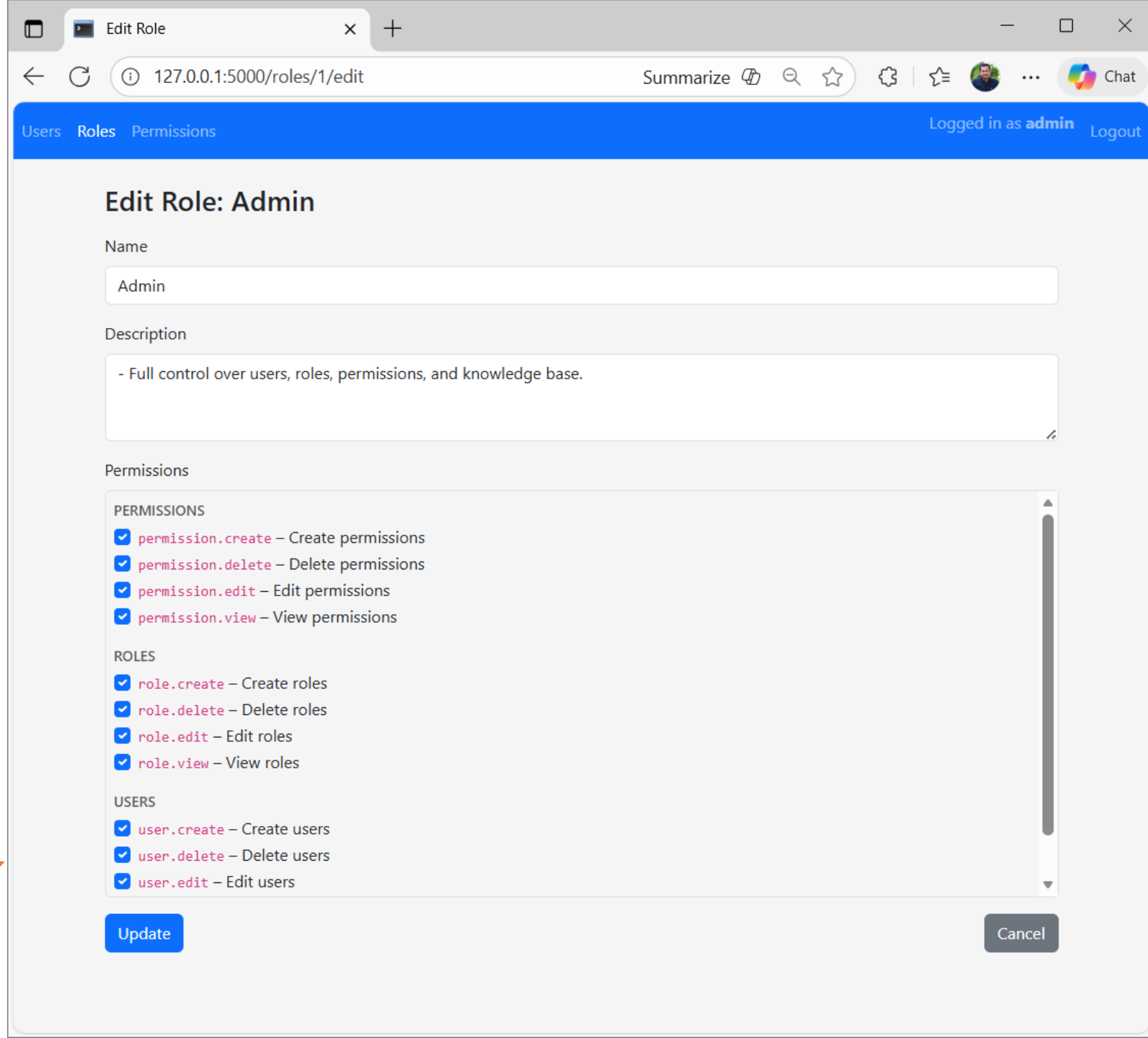
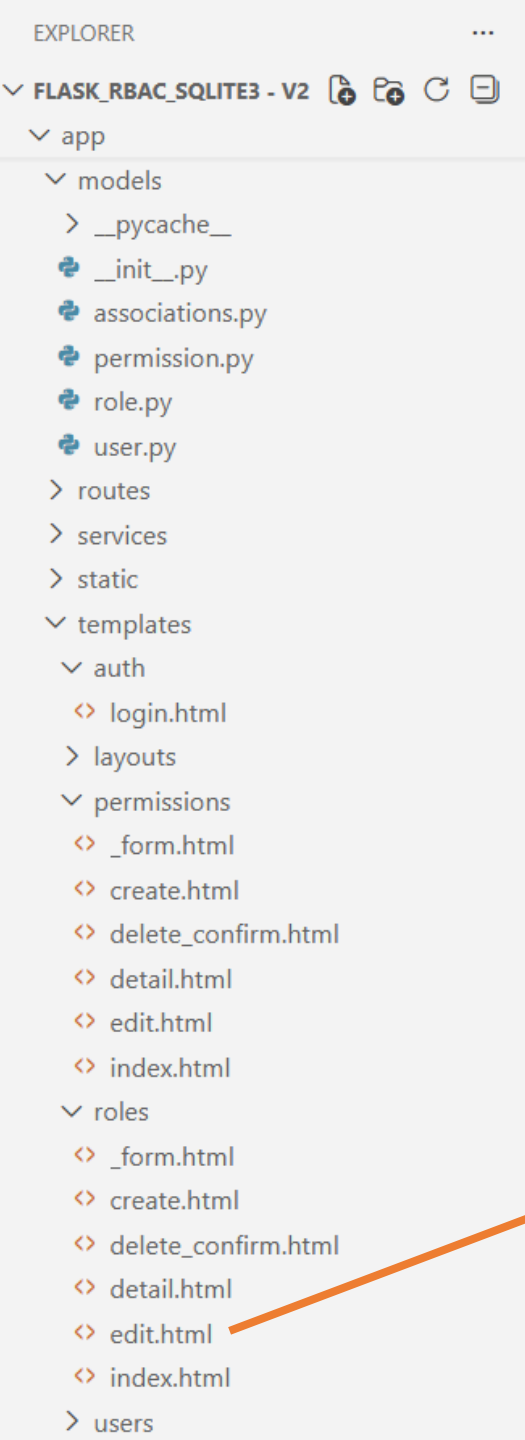


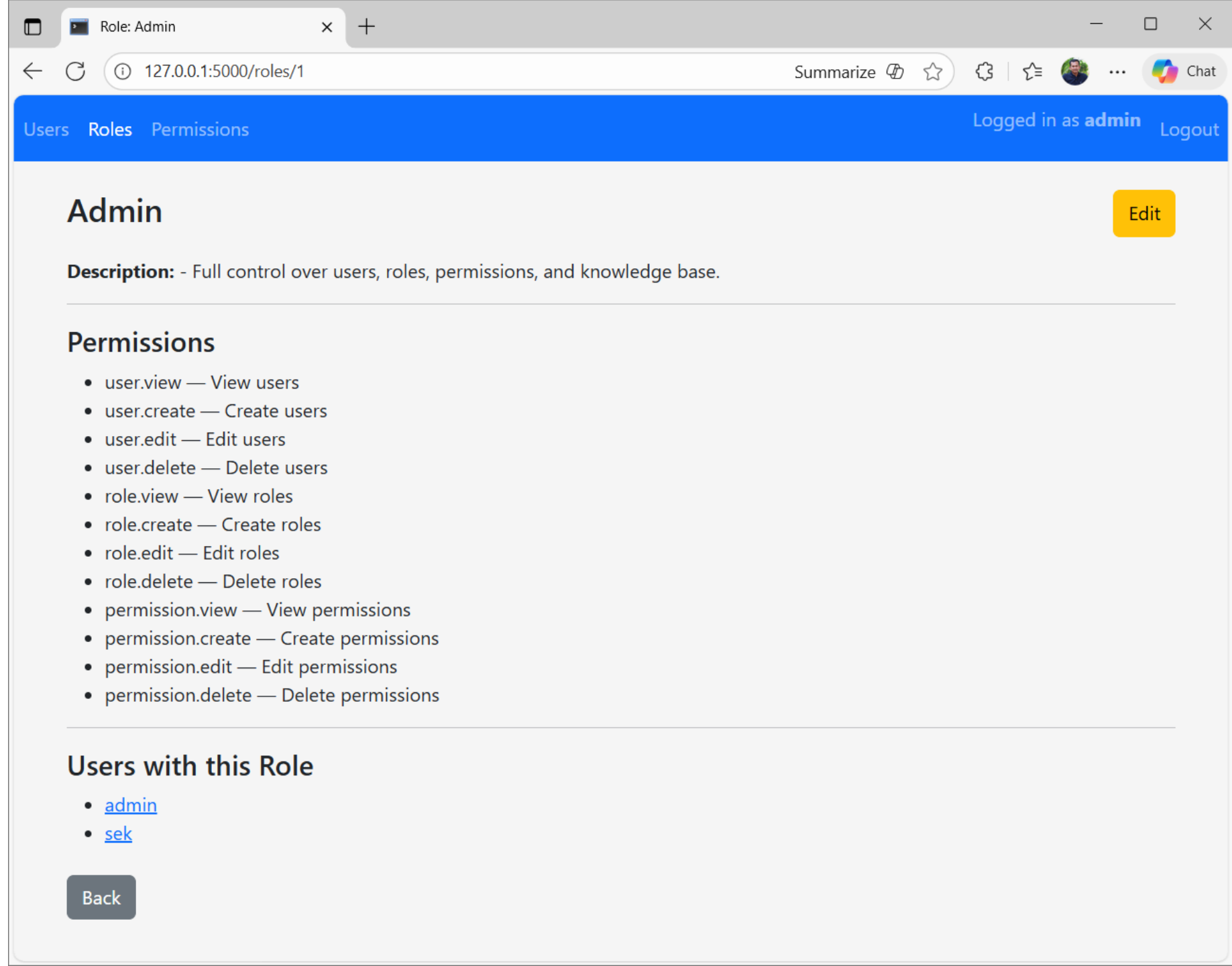
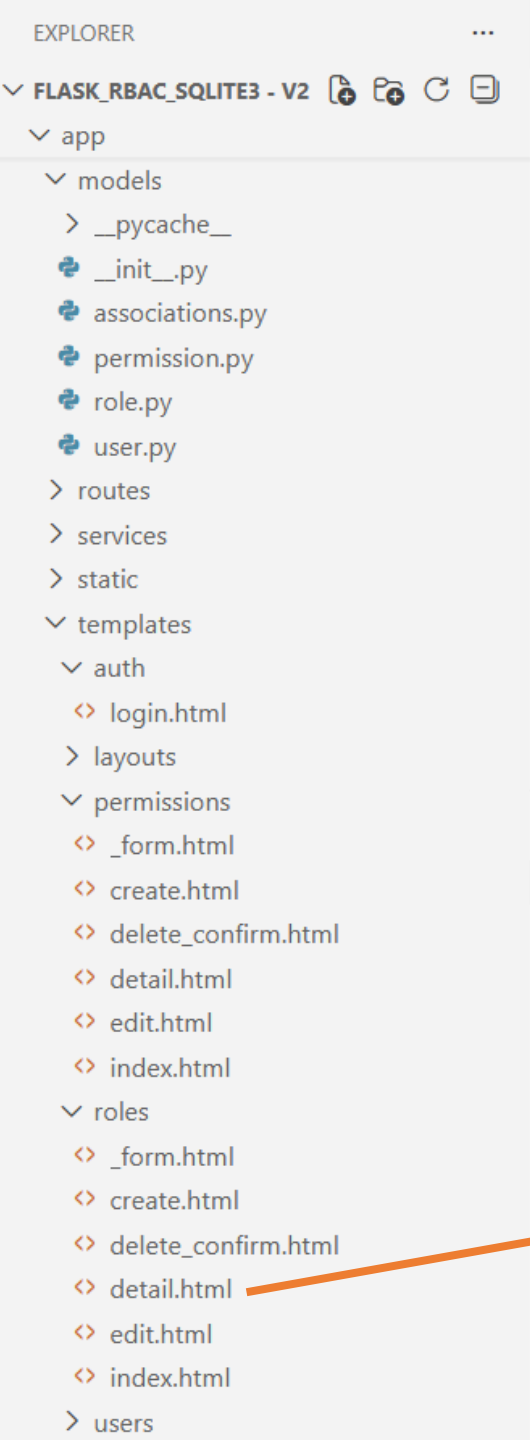
3 — GUI in templates/roles

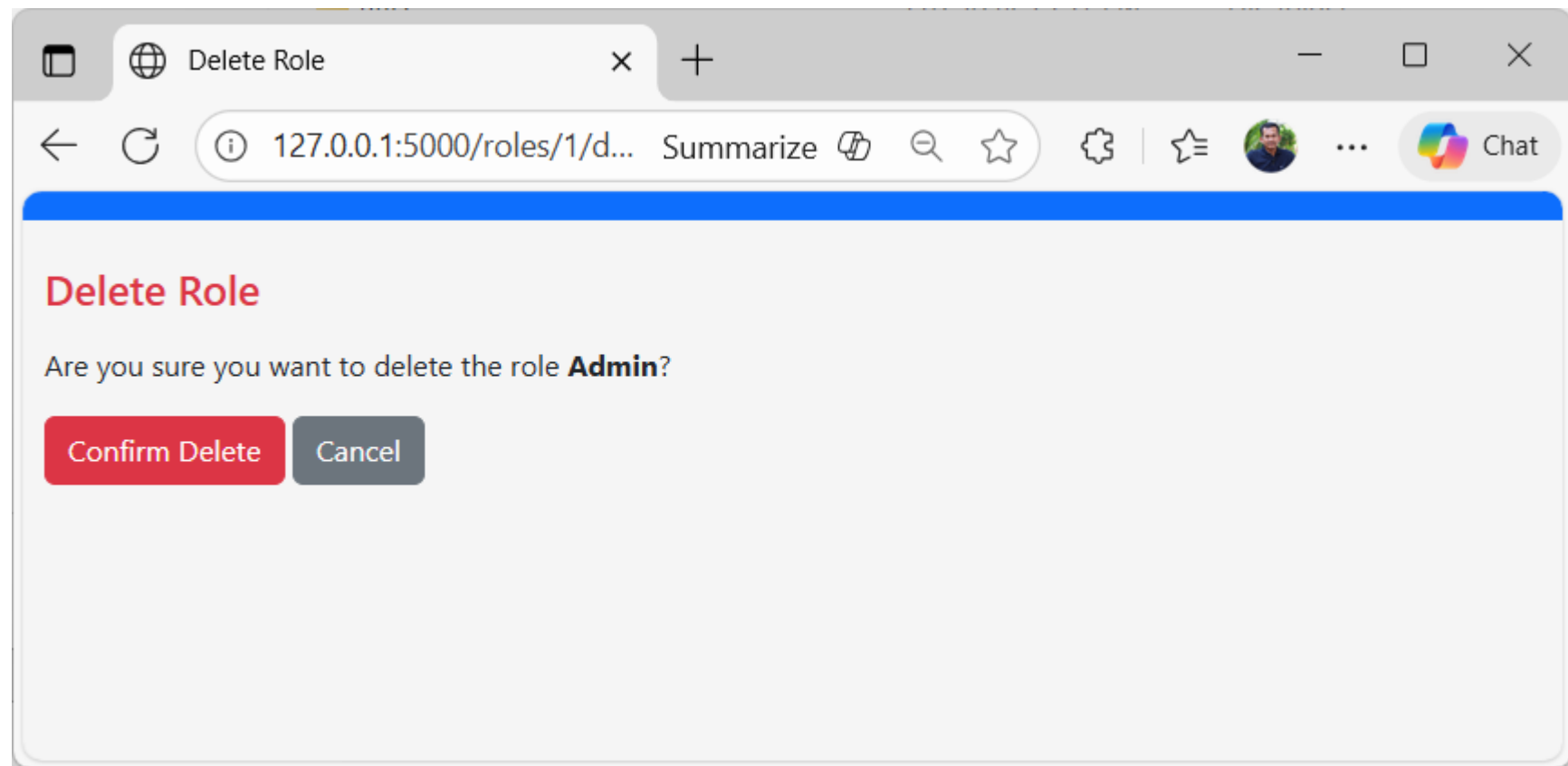
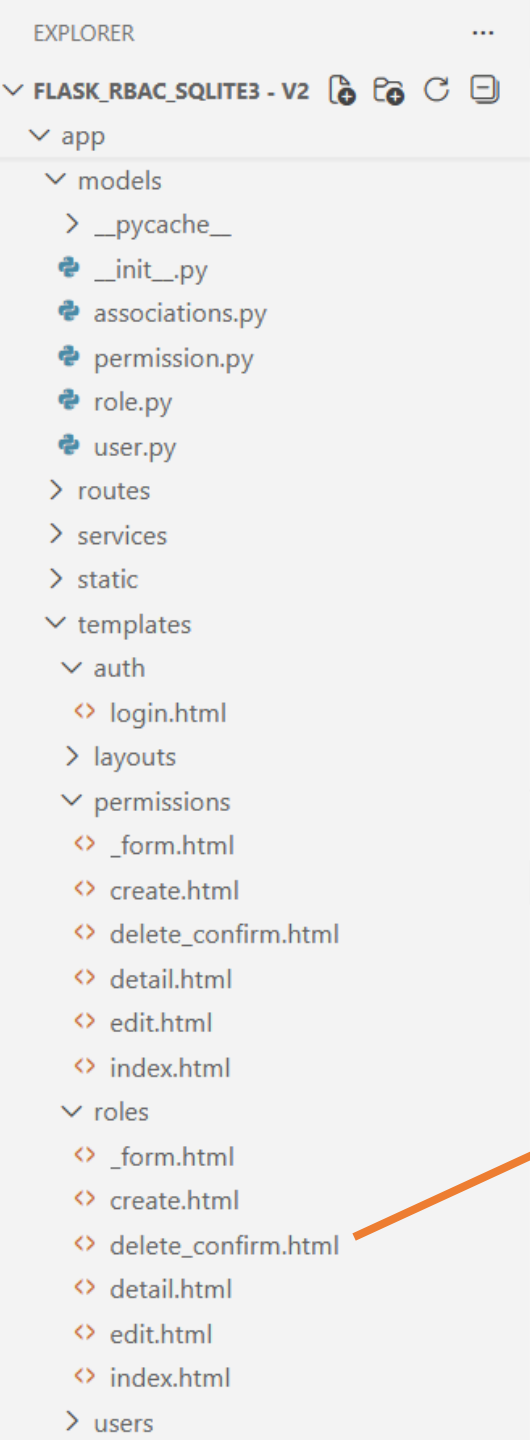






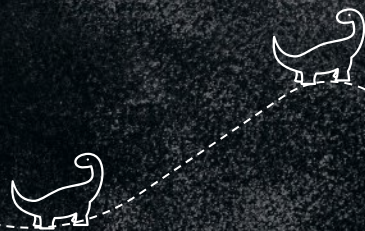


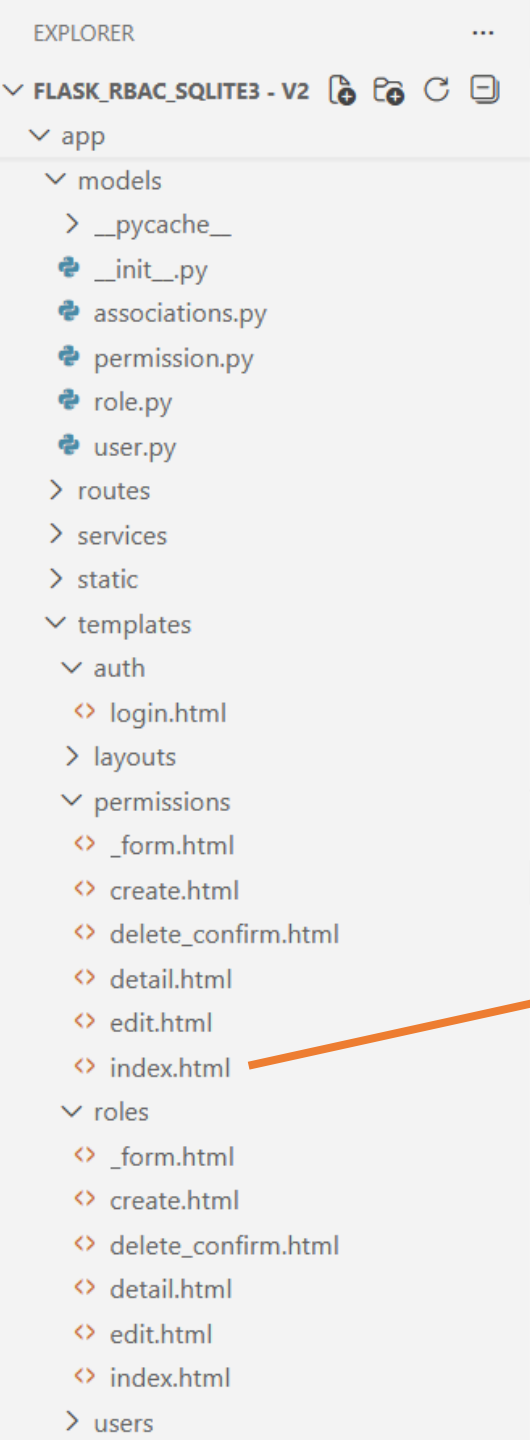






4 — GUI in templates/permissions





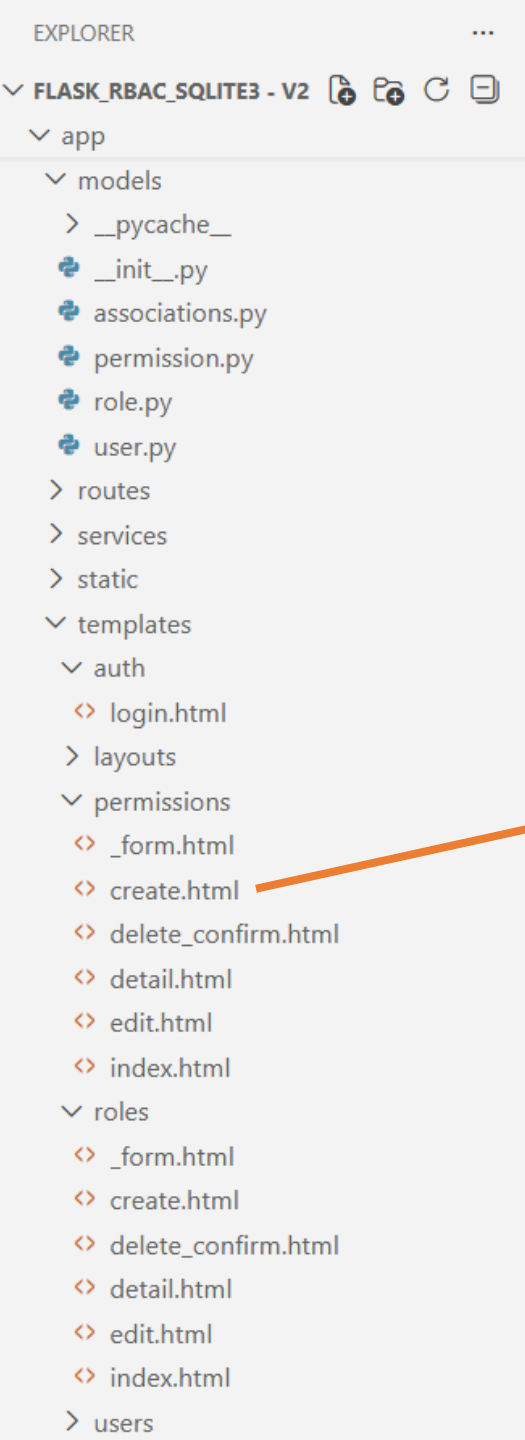
Permissions

127.0.0.1:5000/permissions/

Summarize

Create Permission

Code	Name	Module	Description	# Roles	Actions
permission.create	Create permissions	Permissions	Can create new permissions	4	 
permission.delete	Delete permissions	Permissions	Can delete permissions	2	 
permission.edit	Edit permissions	Permissions	Can edit existing permissions	3	 
permission.view	View permissions	Permissions	Can see permission list and details	5	 
role.create	Create roles	Roles	Can create new roles	4	 
role.delete	Delete roles	Roles	Can delete roles	2	 
role.edit	Edit roles	Roles	Can edit existing roles	3	 
role.view	View roles	Roles	Can see role list and details	5	 
user.create	Create users	Users	Can create new users	4	 
user.delete	Delete users	Users	Can delete users	1	 
user.edit	Edit users	Users	Can edit existing users	3	 
user.view	View users	Users	Can see user list and details	5	 



Create Permission

127.0.0.1:5000/permissions/create

Create Permission

Code

e.g. user.view

Name

Human-readable name

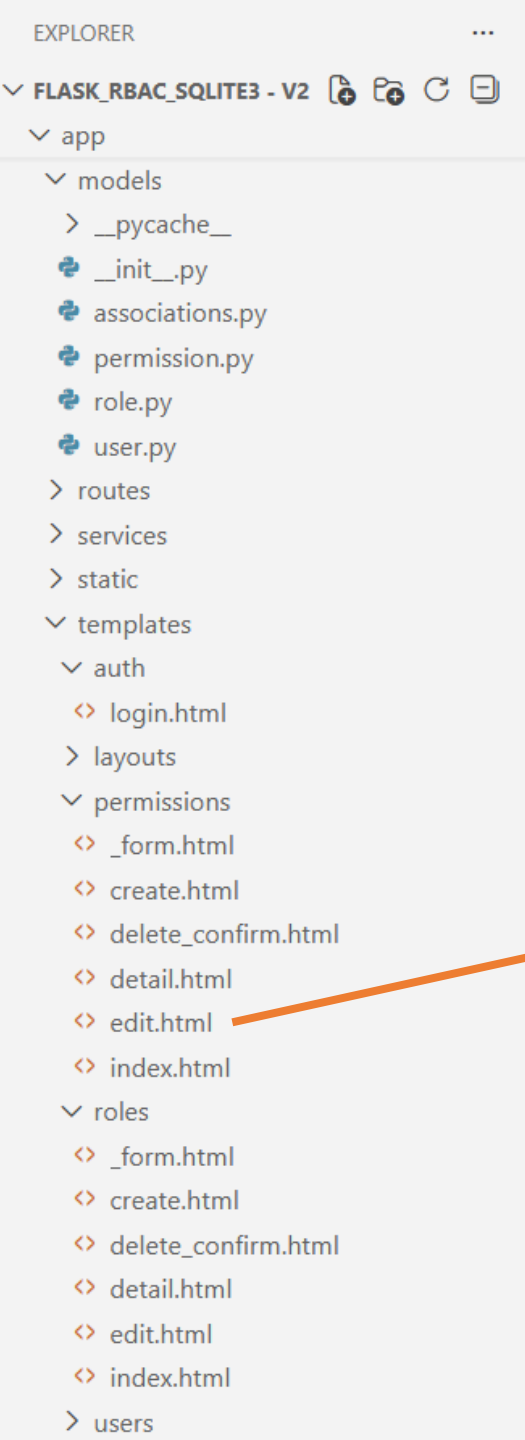
Module

General

Description

What does this permission allow?

Save Cancel



Edit Permission

127.0.0.1:5000/permissions/10/edit

Summarize

Edit Permission: permission.create

Code

permission.create

Name

Create permissions

Module

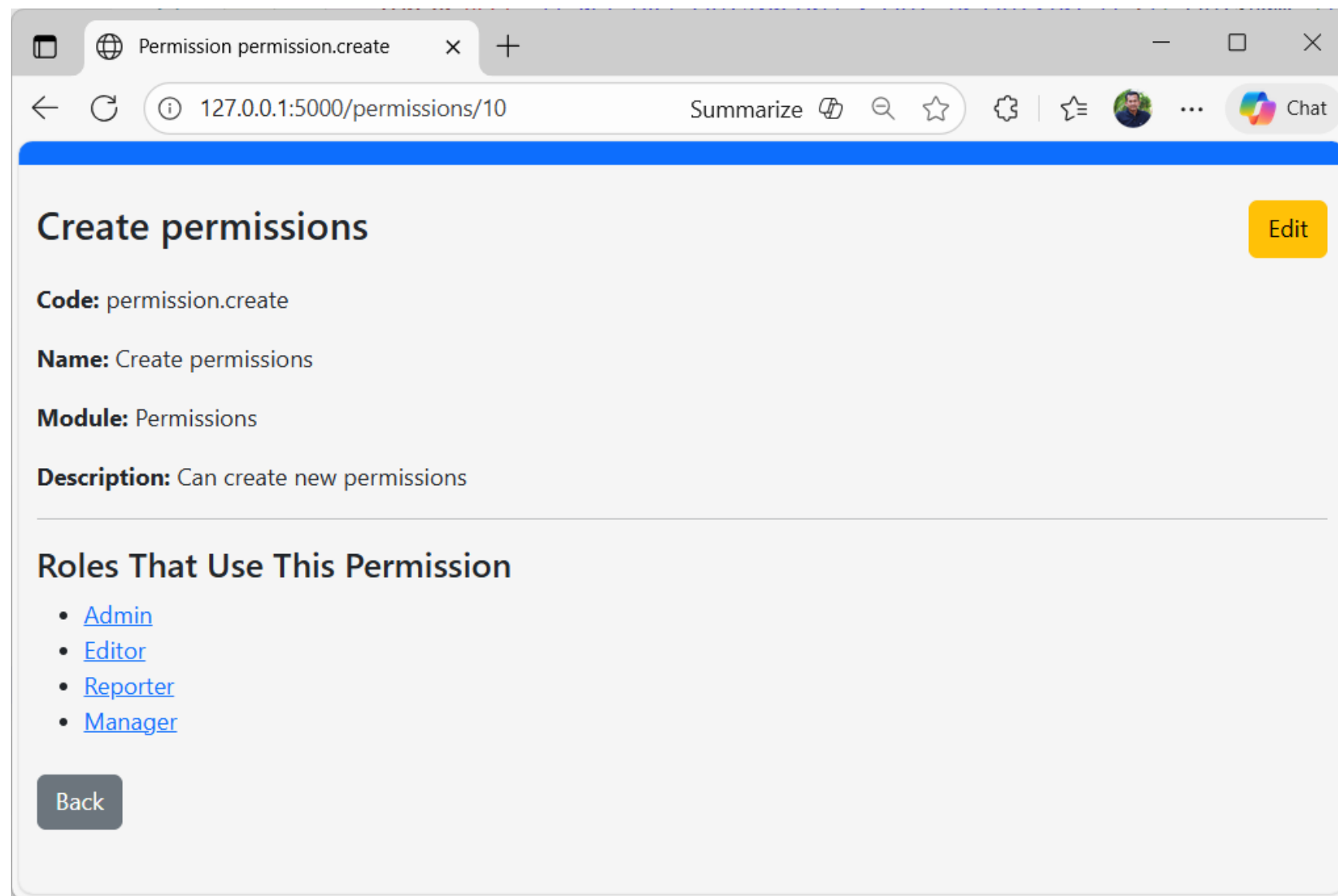
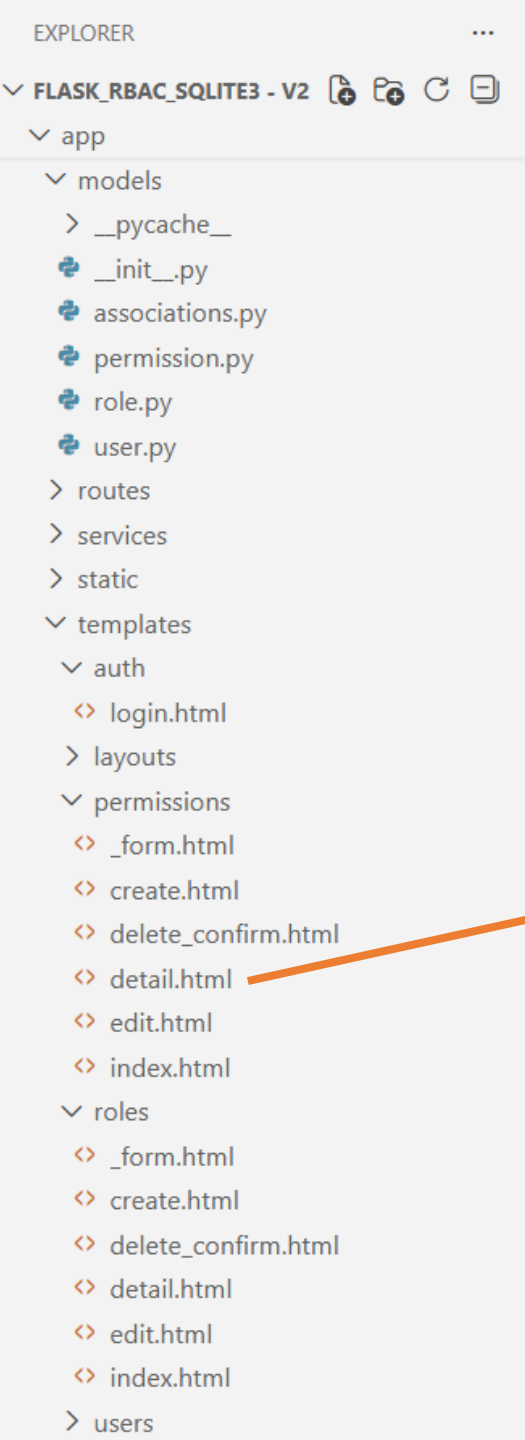
Permissions

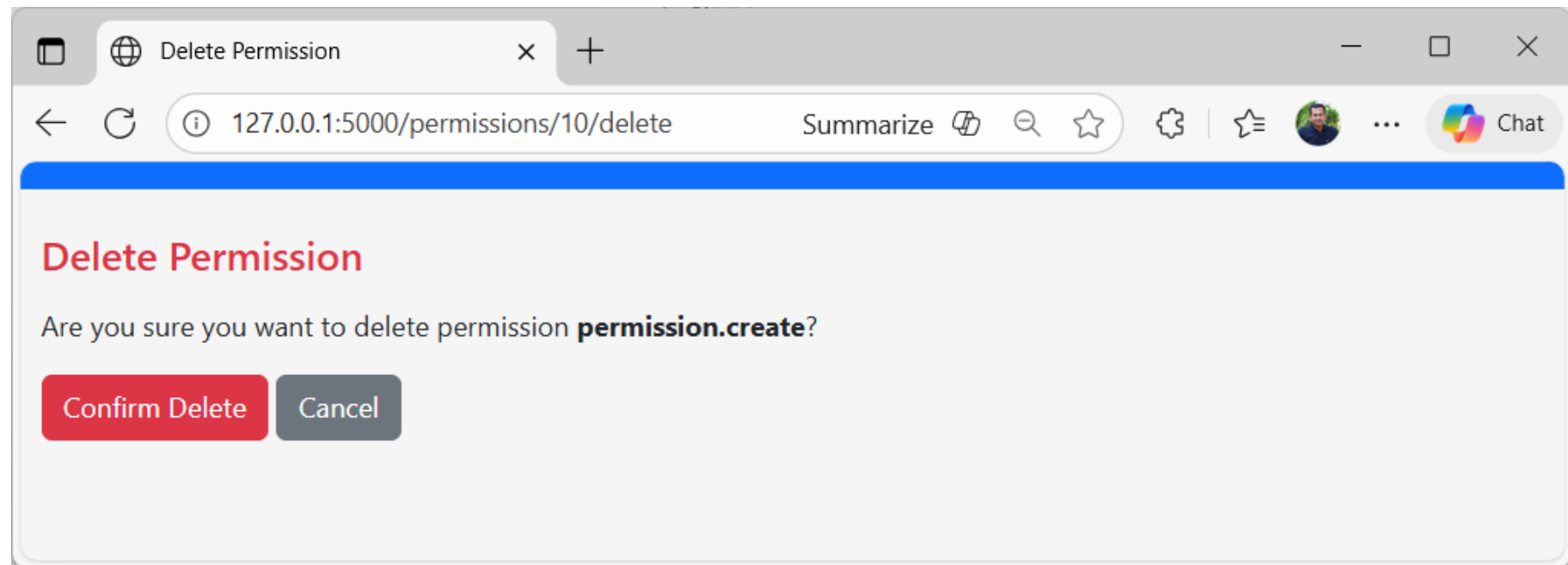
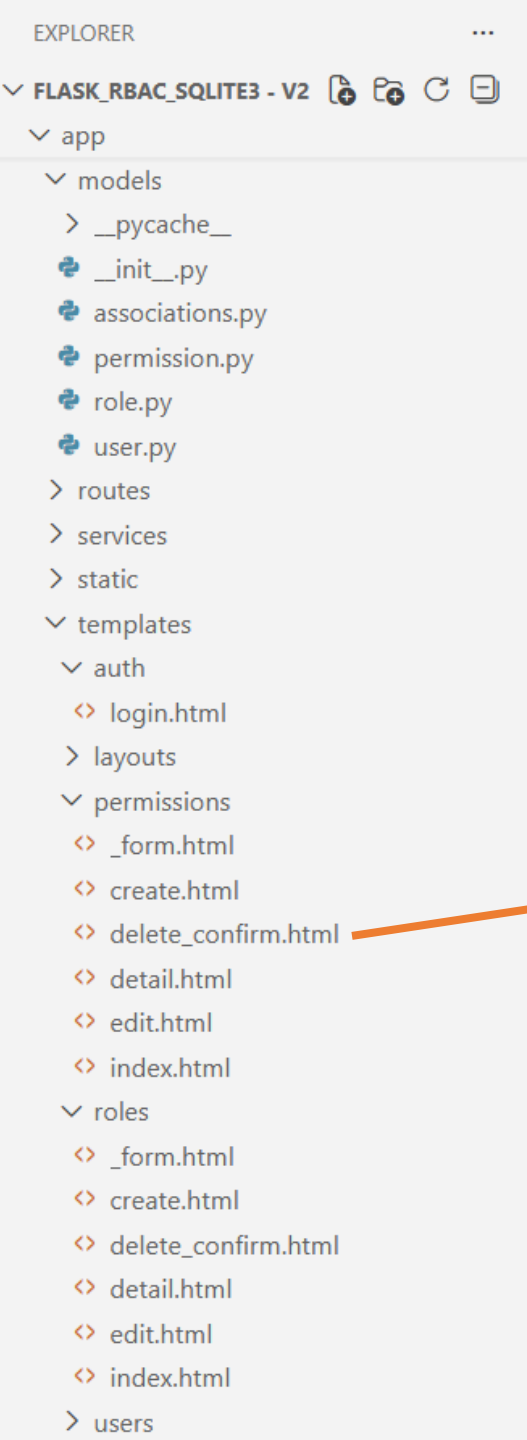
Description

Can create new permissions

Update

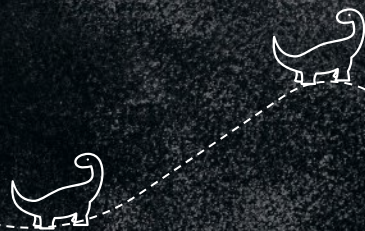
Cancel

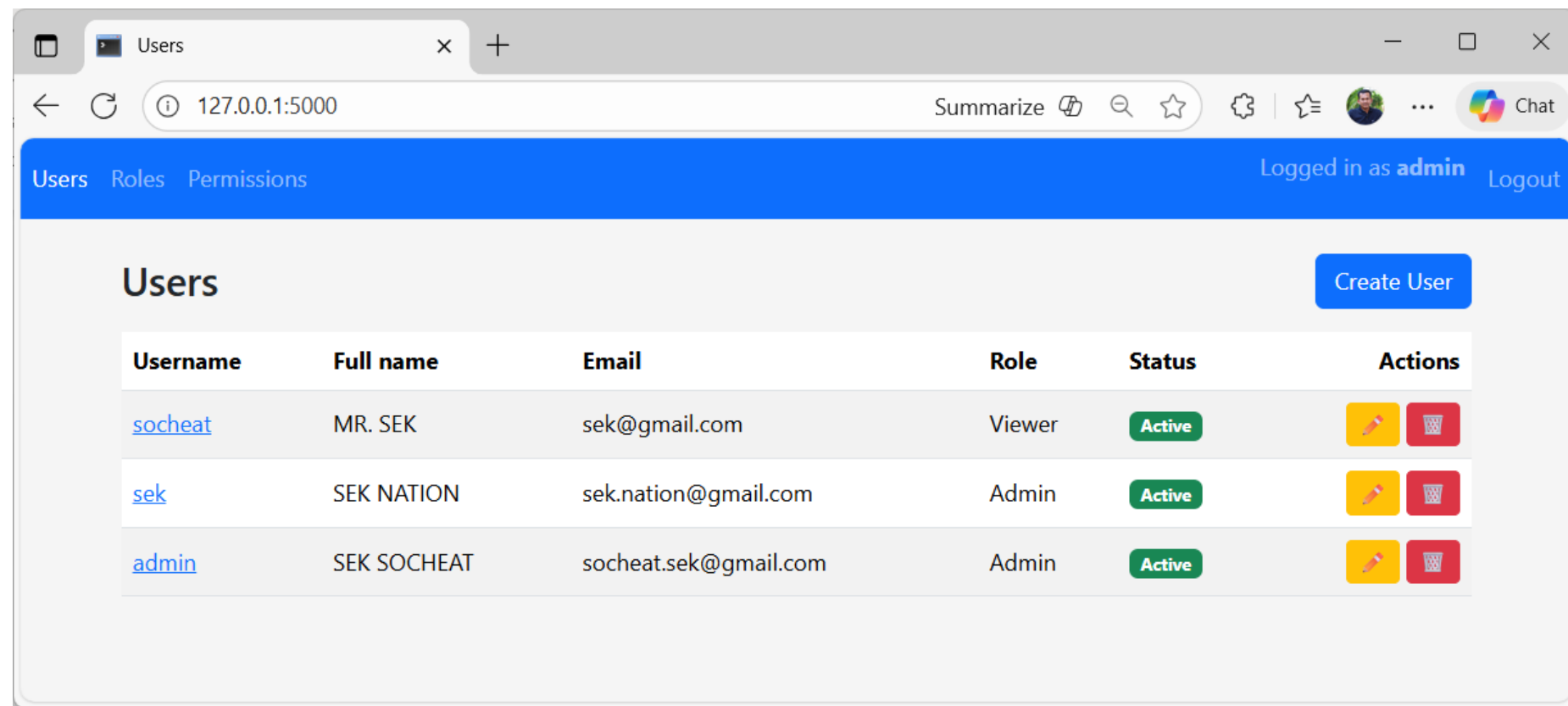
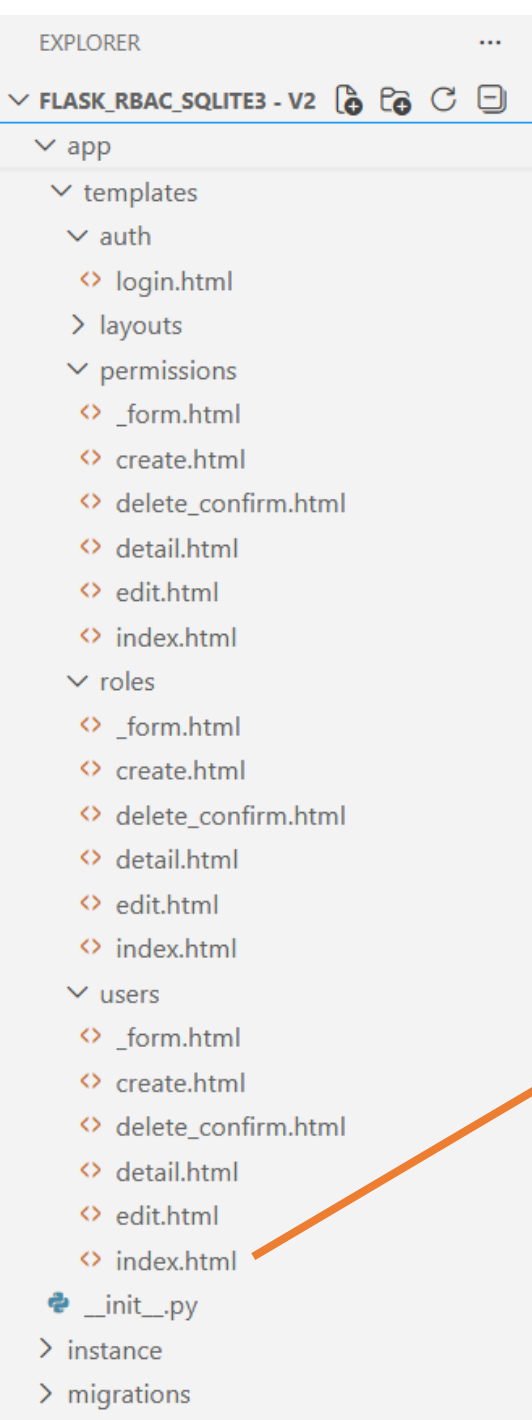


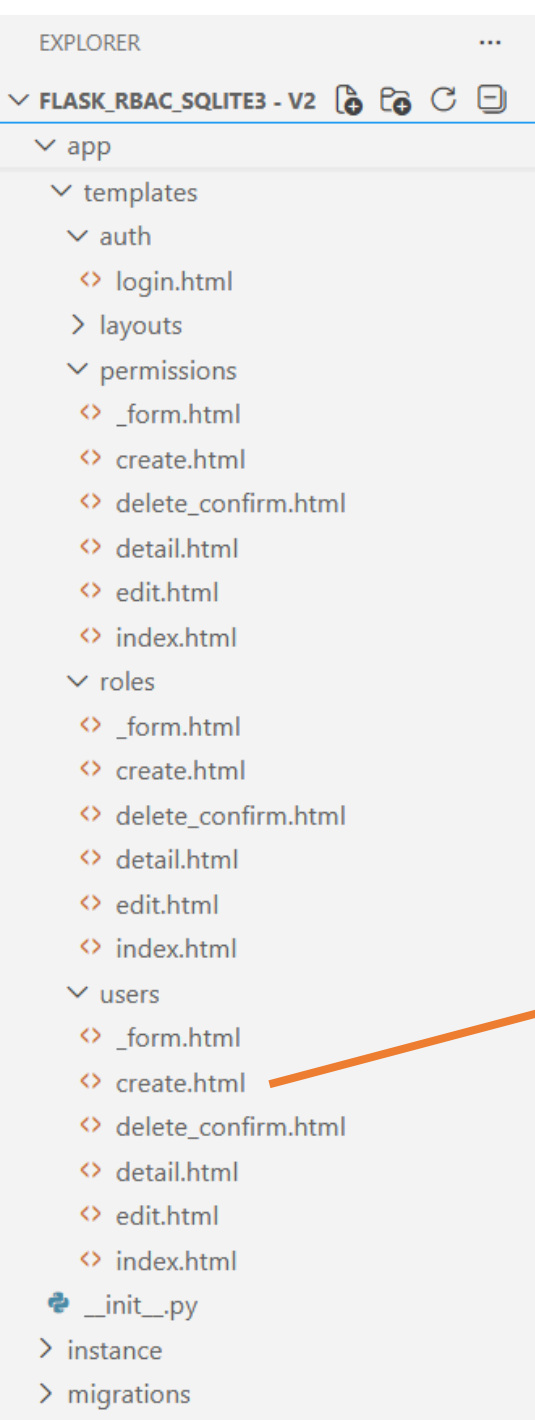




5 — GUI in templates/users







Create User

127.0.0.1:5000/create

Summarize 🔗 🔍 ☆ ⚙️ | ☆ 👤 ... Chat

Users Roles Permissions

Logged in as **admin** Logout

Create User

Username

Email

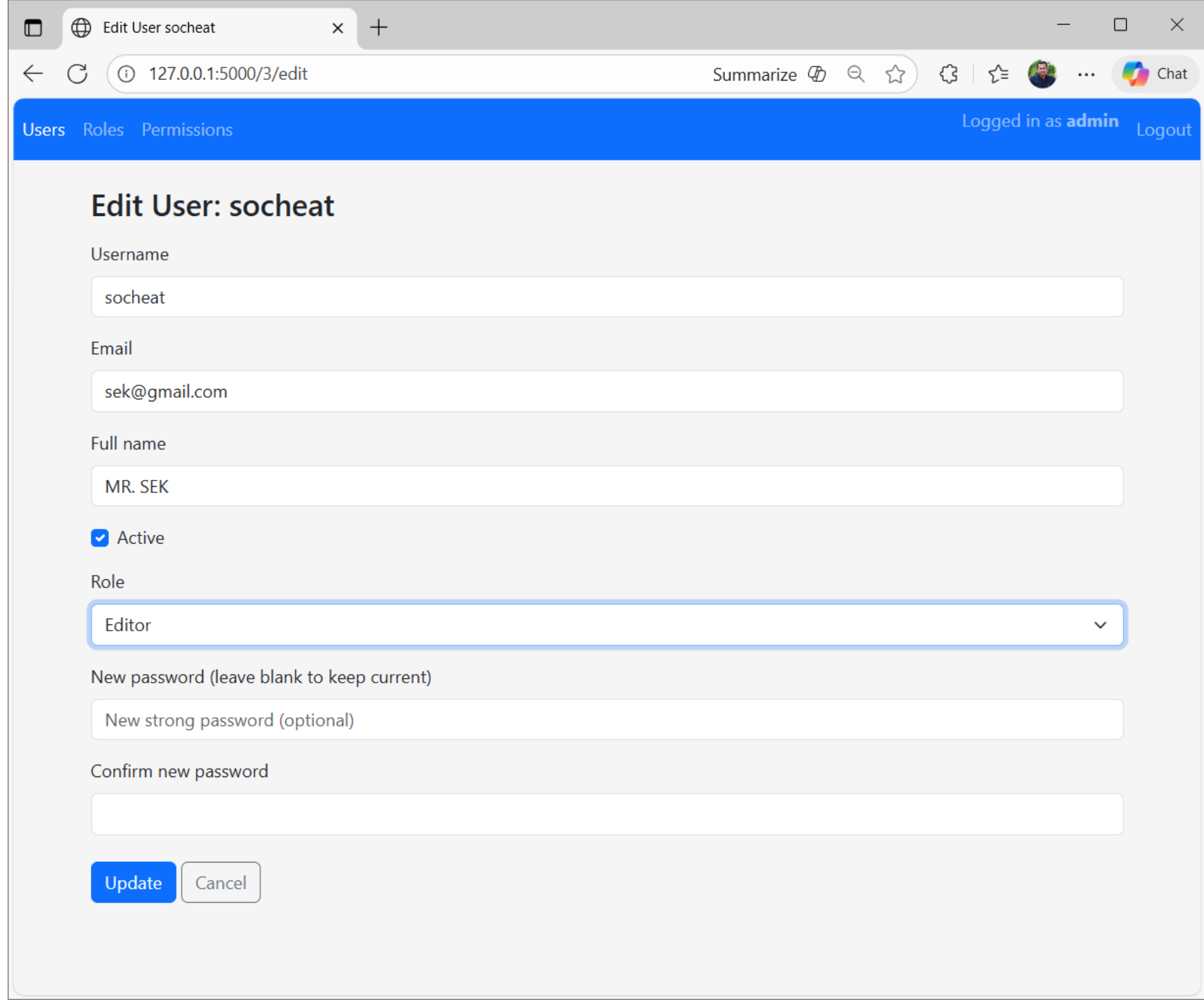
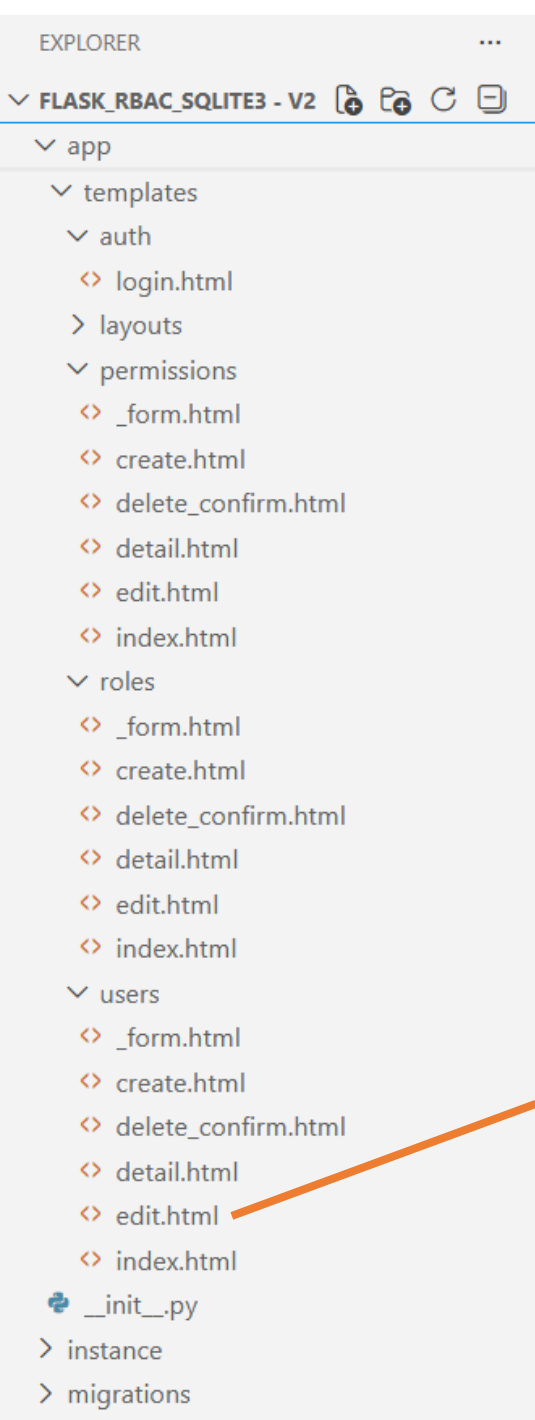
Full name

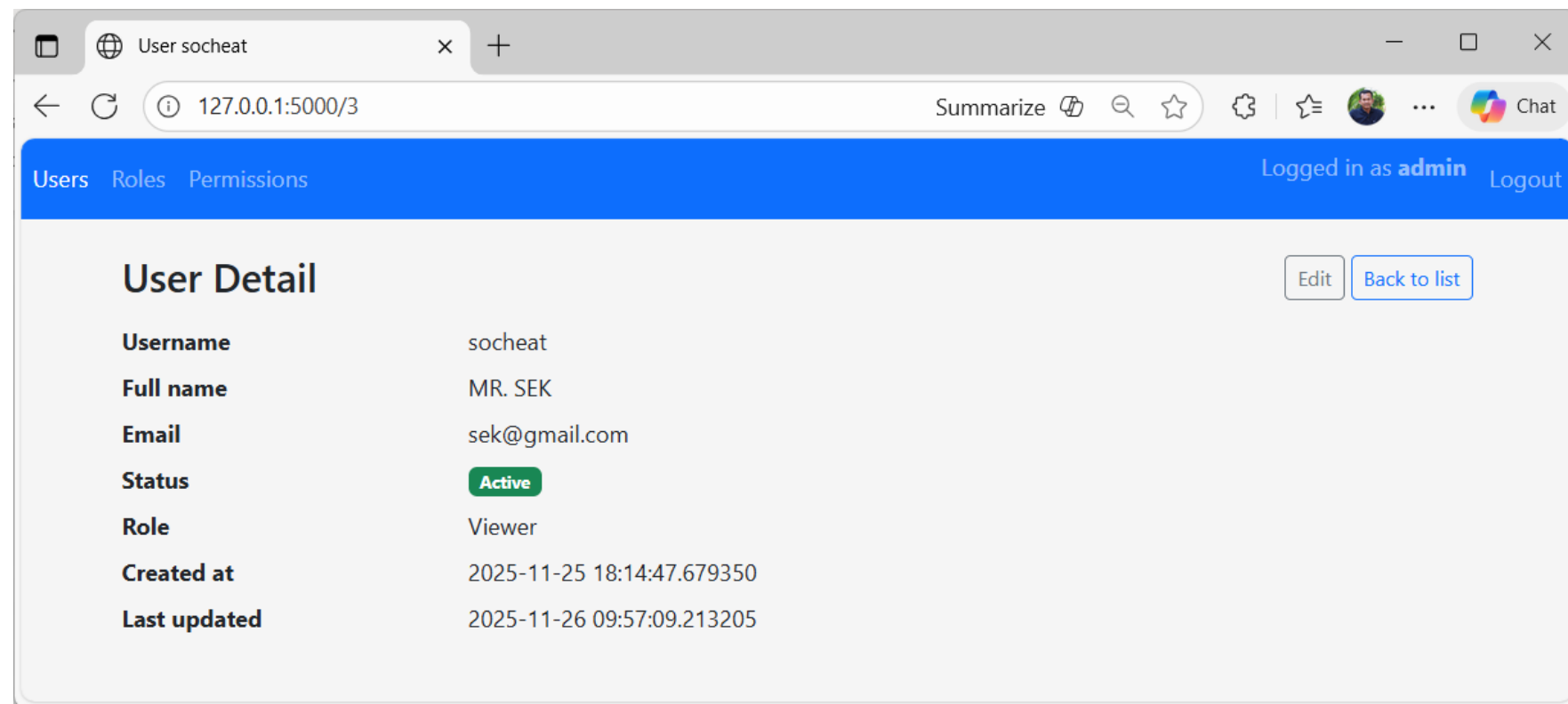
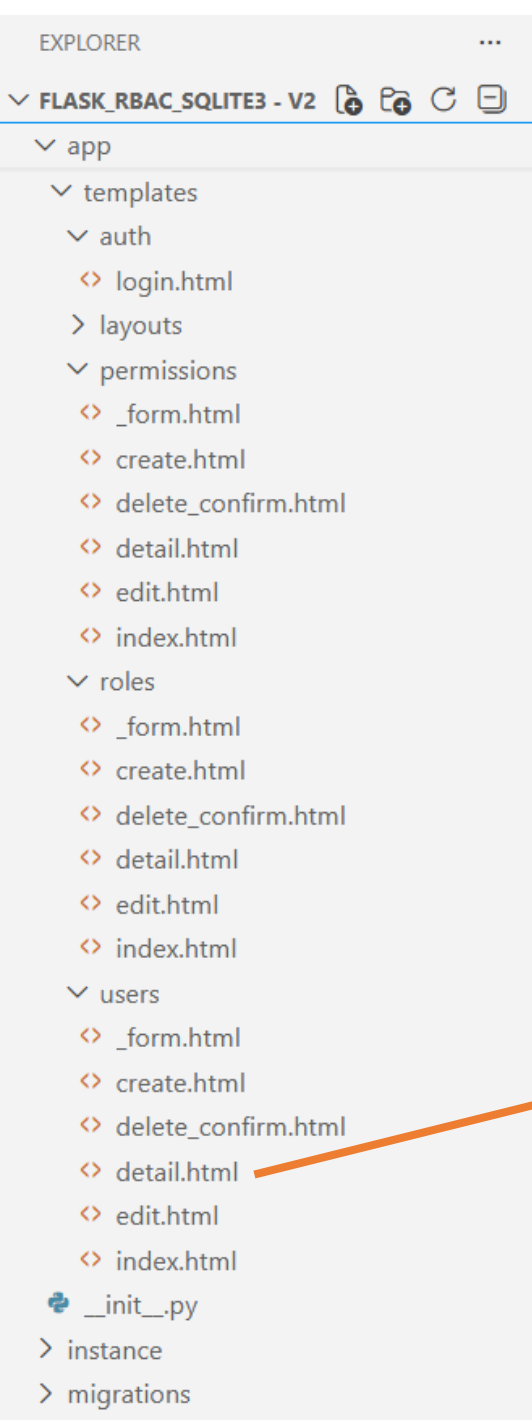
☒ Active

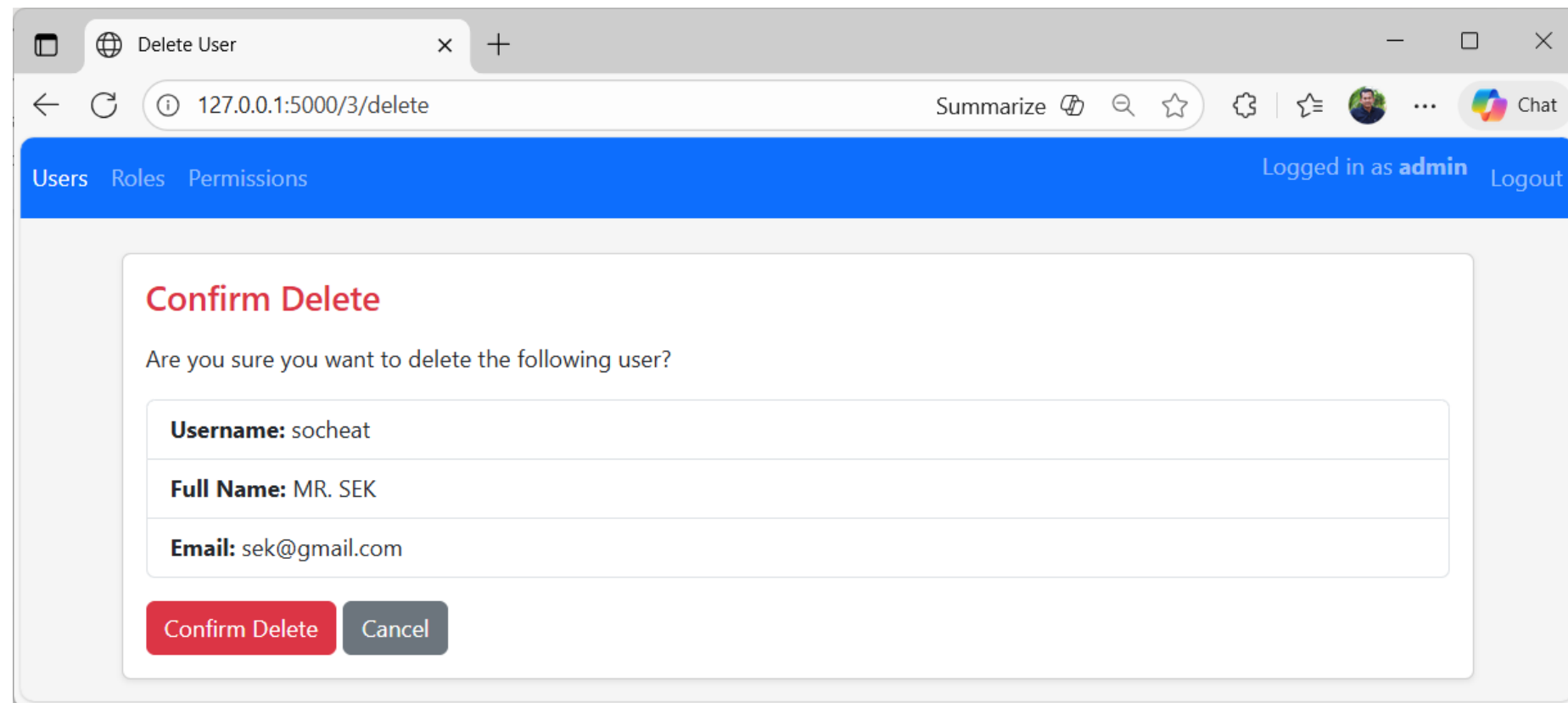
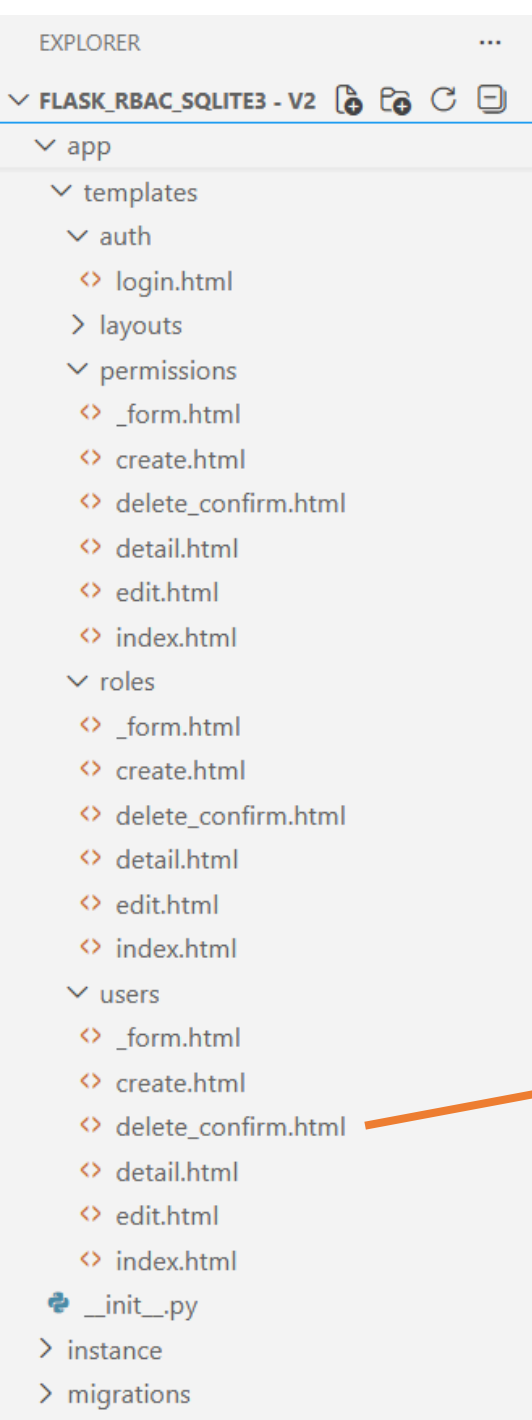
Role

Password

Confirm password

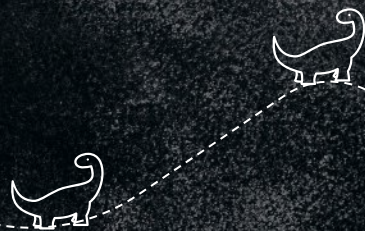


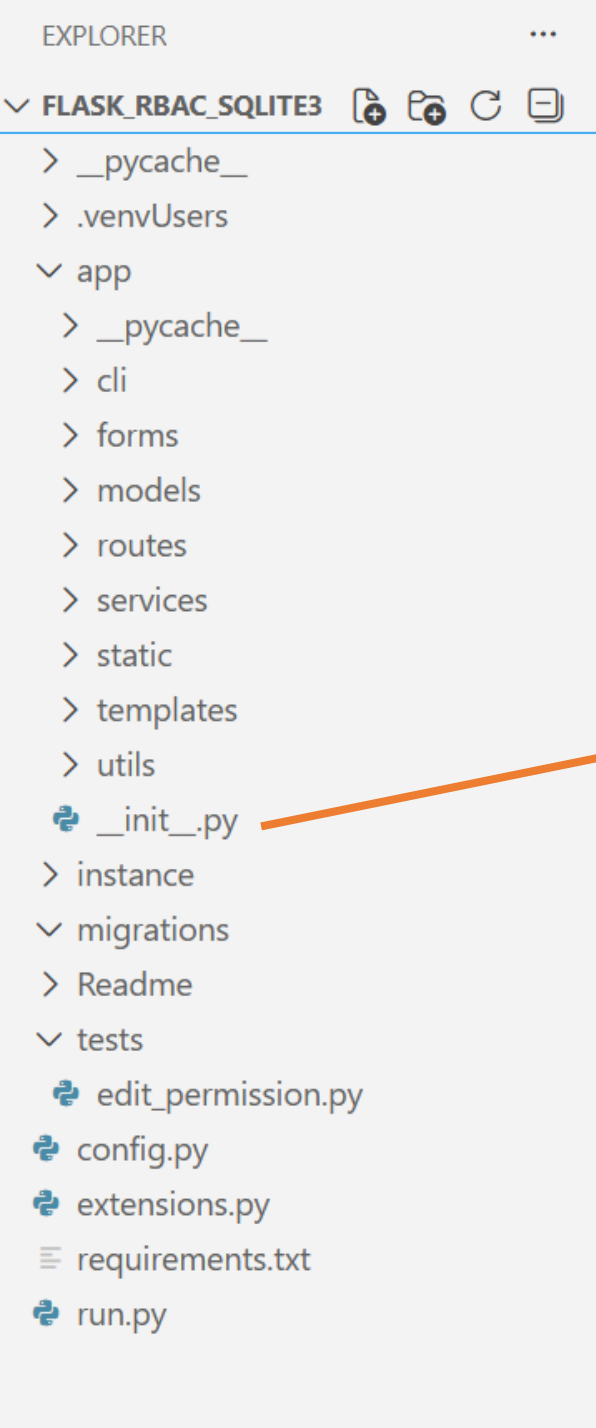






6 — Update app/___init___.py





```
app > 🔗 __init__.py > ...
1  # app/__init__.py
2  from flask import Flask, redirect, url_for
3  from config import Config
4  from flask_login import LoginManager
5  from extensions import db, csrf, login_manager
6  from app.models.user import User
7
8  def create_app(config_class: type[Config] = Config):
9      app = Flask(__name__)
10     app.config.from_object(config_class)
11
12     db.init_app(app)
13     csrf.init_app(app)
14     login_manager.init_app(app)
15
16     # Optional settings
17     login_manager.login_view = "auth.login" # Blueprint.route name
18     login_manager.login_message = "Please log in to access this page."
19     login_manager.login_message_category = "warning"
20
21     # This function tells Flask-Login how to load a user from a session
22     @login_manager.user_loader
23     def load_user(user_id):
24         return User.query.get(int(user_id))
25
26     # Register blueprints
27     from app.routes.user_routes import user_bp
28     from app.routes.role_routes import role_bp
29     from app.routes.permission_routes import permission_bp
30     from app.routes.auth_routes import auth_bp # Optional login/logout routes
31
32     app.register_blueprint(user_bp)
33     app.register_blueprint(role_bp)
34     app.register_blueprint(permission_bp)
35     app.register_blueprint(auth_bp)
36
37     @app.route("/")
38     def home():
39         return redirect(url_for("users.index"))
40
41     with app.app_context():
42         from app.models import User, Role, Permission
43         db.create_all()
44
45     return app
```

Update VENV:

`pip install Flask-Login`



Thank You

SEK SocheaT

 socheatsek@norton-u.com

